IOS · SWIFTUI · SWIFTDATA

Winter Arc: the decisions behind the build

A habit tracker for a 90-day, time-boxed challenge. One hard start, one hard end, a handful of daily non-negotiables. This is a record of the engineering decisions, including the ones that were wrong and had to be taken back out.

Shipped v1.0 29 Sep 2026 · v1.1 3 Oct 2026 **Scope** 35 functional requirements, 10 non-functional

Dependencies zero third-party

Foundations

Four decisions taken before implementation that shaped everything after them.

Specifications are locked before code, and conventions are enforced as tests

KEPT

DECISION

Five spec documents define requirements, data model, screens and test cases. Every rule that could be silently violated later became a lint test that greps the source and fails the build: `CONV-01` (no wall-clock reads), `CONV-02` (no networking), `CONV-04` (inflection markup only in `Text` literals), `CONV-05` (motion only through the Reduce Motion helpers), `CONV-06` (no colour literals outside `Palette.swift`).

WHY

A convention written in a document is advice. The same convention written as a failing test is a constraint. Three of these tests exist because the rule was broken first and the bug shipped.

COST — Lint tests can pass vacuously. One of them did — see the measurement section.

Dates are calendar strings, never timestamps

KEPT

DECISION — A day is a `DateKey` — `yyyy-MM-dd` produced by an explicit Gregorian calendar with an injected time zone. Entries are keyed by that string. No `Date` is stored for a check-in.

WHY — "Did I do this on Tuesday?" is a calendar question, not an instant-in-time question. Storing timestamps means DST transitions, travel across time zones and midnight boundaries each become a correctness bug. Storing the day as a string means they become nothing at all.

COST — Date arithmetic has to go through a small typed layer instead of `Date`. Worth it: the test suite includes DST-forward, DST-back, time-zone travel and clock-rollback cases, and they pass by construction.

Day rollover is lazy, cursor-driven and idempotent

KEPT

DECISION — No background task. On launch and every foreground, a startup gate processes every date after `rolloverCursorDateKey` up to yesterday, marks missing entries missed, applies ruleset effects and advances the cursor. Running it twice on the same day is a no-op.

WHY — Background execution on iOS is not guaranteed, so anything that depends on it is a class of bug you cannot reproduce. A cursor plus idempotency means the app is correct whether it was opened at midnight, after a week, or after ninety days.

COST — The worst case — a 90-day catch-up across 7 habits, 630 inserts in one save — happens while the user waits. Measured at **`0.228 s`** against a **`1 s`** budget.

The widget reads a JSON snapshot, not the database

KEPT

DECISION

After every write the app serialises a small snapshot to `widget-snapshot.json` in the App Group container, written atomically. The widget extension opens that file and never touches SwiftData.

WHY

Two processes sharing a store is a concurrency problem with no upside here — the widget shows a day index and a done count. A file read cannot corrupt anything, cannot block, and degrades to a defined empty state when missing or unreadable.

COST

A second representation of the same data to keep in sync, and a schema version in the file so an unknown one renders the empty state rather than wrong numbers.

Where the framework fought back

Three cases where the obvious approach was wrong, and the fix was only discoverable by running it.

SwiftData relationship mutations do not notify SwiftUI

WORKED AROUND

PROBLEM

Toggling a habit writes through a relationship. Views deriving state from those entries kept rendering stale data — the checkbox moved, the consistency score did not.

FIX

An observable `dataVersion` counter on the app environment, incremented after any write that can change arc state. Views that derive from entries read it to make the dependency explicit.

JUDGEMENT

This is a workaround, not a design, and it is commented as one. The alternative — restructuring the model so every derived view observes a value type — was a larger change than the bug justified.

A background set above `TabView` is painted over by it

REVERSED

PLAN

The visual restyle specified one backdrop gradient drawn once by the root container, with every screen transparent over it.

WHAT HAPPENED

Every screen went flat white. `TabView` and `NavigationStack` paint their own background over anything set above them; the gradient has to sit on the screen's own content, inside the stack.

RESOLUTION

Reverted to one call per screen — fifteen sites. Sheets and full-screen covers are separate window layers and need their own regardless, so per-screen turns out to be the uniform rule rather than a special case. The spec now records the failed approach and why, so it is not re-attempted from the document.

`List` rows keep the system background

REVERSED

ATTEMPT

To carry the new card styling into Settings and the other list screens, rows were given `.listRowBackground` with the custom fill.

WHAT HAPPENED

It flattened the inset-grouped geometry into full-bleed rows. The screen stopped looking native and gained nothing but a tint.

RESOLUTION

Rows keep the system's own inset-grouped background; only `.scrollContentBackground(.hidden)` is applied so the gradient shows between sections. Losing the platform's row shape cost more than the visual consistency gained.

A bug class that shipped three times

The same mistake in three places, each found separately, before the cause was named and made impossible.

Hardcoded light colours on tokens that invert

FIXED

INSTANCES

A button gradient built from sRGB literals, which silently discarded the accent's dark-mode and high-contrast variants. A white button label on an

accent that inverts, measuring about **2.6:1** in dark mode. A white checkmark on the done-state green, measuring **1.88:1** in dark mode against a **3:1** floor — this one shipped to the App Store and stayed there for five days.

ROOT CAUSE

A literal cannot carry appearance variants. Every one of these looked correct in the appearance it was written in.

FIX

Colours resolve from the asset catalogue through a single `Palette.swift`. Foreground colours that sit on an inverting fill use `systemBackground`, which inverts with it — the checkmark went from **1.88:1** to **11.19:1** in dark, **5.34:1** in light. `CONV-06` now fails the build on a colour literal anywhere else, and two regression tests pin the old broken values below their thresholds so they cannot return quietly.

Contrast is measured at runtime, not asserted from the design

KEPT

DECISION

Contrast tests resolve the real asset-catalogue colours under each appearance and contrast setting and compute WCAG ratios, rather than checking values copied from a design file.

WHY IT MATTERED

The spec's own colour table had drifted: four values had been darkened in the catalogue during an accessibility pass and never written back. A test comparing against the document would have passed while the app was different.

WHAT IT CAUGHT

A streak colour taken from the mockup measured **3.85:1** on its surface — below AA for text. Darkened to **5.11:1** before it shipped. It also disproved a claim in the spec that a system label "passes by construction": it measures **3.44:1** on pure white and never passed.

Measurement

Three cases where the instrument was wrong, and one where the result was green for the wrong reason.

A green test suite that ran nothing

CAUGHT

PROBLEM A new lint test passed. Checking whether it would actually fail — by planting a violation — it still passed. On a clean build, it still passed.

CAUSE `xcodebuild -only-testing` with a name that matches nothing runs zero tests and prints `** TEST SUCCEEDED **`. The filter named a suite that did not exist.

PRACTICE SINCE Every lint test in this project is verified by planting a violation and watching it fail. A filtered green run proves nothing until the filter is known to resolve.

The launch-time instrument was measuring itself

REPLACED

PROBLEM `XCTApplicationLaunchMetric` reported about **`1.3 s`** against a **`1 s`** budget. A Release build came back slower and noisier, which is the opposite of what a real regression does.

FIX The app measures itself: from the kernel's process start time — so dyld and pre-main work are included — to the first interactive frame, printed to the console in DEBUG. Median across cold launches: **`0.374 s`**.

What the performance suite does not cover

STATED

POSITION The three performance tests are data-layer measurements. They were re-run after the visual restyle and were unchanged — but a gradient was never going to affect how fast 630 rows insert.

WHY IT IS WRITTEN DOWN The restyle's real risk is a translucent fill and a shadow on every card down a long scrolling list, and nothing in the suite exercises rendering. Three green tests would otherwise read as coverage. The spec states the gap explicitly rather than letting the suite imply it.

MEASUREMENT	BUDGET	RESULT
Worst-case rollover, 630 habit-days in one save	`< 1 s`	`0.228 s`

MEASUREMENT	BUDGET	RESULT
All metrics over a 180-day, 7-habit arc	< 100 ms	0.005 s
Startup gate: store open plus first rollover	< 300 ms	0.007 s
Cold launch to interactive, median	< 1 s	0.417 s
Accessibility audit findings	—	67 → 53
Dynamic Type audit at AX5, eight screens	0 findings	clean

Letting evidence overrule the plan

The first week of production data changed what the next release is.

A first-run problem wearing the costume of a product problem

RESCOPED

SIGNAL

About **27%** of installs deleted the app within a week. Day-1 retention around **7%** against roughly **25%** typical for iOS. But the users who stayed opened it about **3.5** times a day.

READING

A product failing its users does not produce survivors that engaged. People were not bouncing off the daily loop — most never reached it. Between download and loop sat seven setup screens, and the daily reminder defaulted off, so anyone who abandoned setup also left with nothing to bring them back.

CONSEQUENCE

The next milestone had been a visual refresh of the welcome screen. It was rescoped behind a one-tap quick start and a reminder that defaults on. Four onboarding designs had already been drawn; none of them shortened the flow, so none of them addressed what the numbers showed.

Three changes in forty-eight hours, none attributable

MISTAKE

WHAT HAPPENED	A new version, a category change and a keyword rewrite all shipped inside two days. Conversion fell from about **30%** to **10.9%** across the same window.
WHY IT IS RECOVERABLE HERE	The decline began before the new version went live, which rules the screenshots out. That was luck, not method — had the timing been slightly different, the data could not have answered the question it was collected for.
RULE SINCE	One change, one week, then the next.

Interview questions

Questions this project supports, with what a strong answer contains. They work from either side of the table: preparation if you are presenting the work, probes if you are assessing someone on it.

MODELLING AND CORRECTNESS

Why store a check-in against a date string rather than a `Date`?

> LISTEN FOR
> the distinction between an instant and a calendar day; DST, time-zone travel and midnight as the failure modes; and the recognition that this converts a family of runtime bugs into a representation choice.

A user opens the app for the first time in three weeks. Walk me through what happens before the first frame.

> LISTEN FOR
> the startup gate, a cursor advancing one day at a time, idempotency, ruleset effects applied per day rather than in aggregate, and an answer to how long it takes with a number attached.

Grace Days gives a user N forgiven misses. The user edits a past day and that frees one up. What should happen?

> LISTEN FOR
> recognition that failure is a state machine, not a flag: a refund can un-fail an arc only while the failure is unresolved; once the user has chosen to keep going or restart, the status stands. Watch for someone who

> treats it as a simple comparison.

Why does the widget read a JSON file instead of the shared database?

> LISTEN FOR
>
> two processes against one store as the risk being avoided, the widget's needs being trivial, and a defined behaviour when the file is missing, unreadable or an unknown schema version.

FRAMEWORK BEHAVIOUR

You toggle a value through a SwiftData relationship and the view does not update. What is happening and what do you do?

> LISTEN FOR
>
> relationship mutations not reliably publishing; an explicit observable version counter as a pragmatic fix; and — most of all — whether they call it a workaround rather than presenting it as architecture.

You want one background gradient behind every screen of a tabbed SwiftUI app. Where does it go?

> LISTEN FOR
>
> the instinct to put it at the root, then the knowledge that `TabView` and `NavigationStack` paint over it, and that sheets and covers are separate window layers needing their own regardless. A candidate who has only read about this will answer "the root" with confidence.

How would you make an app honour Reduce Motion everywhere, and keep it honouring it a year from now?

> LISTEN FOR
>
> a single seam all motion routes through, a test seam so the setting can be forced in tests, and a lint test banning raw animation elsewhere. The second half of the question is the real one: a policy nothing enforces decays.

ACCESSIBILITY AND COLOUR

A checkmark is white on a green circle and passes review. What would make you check it anyway?

> LISTEN FOR
>
> the question "does the green change between appearances?" If the fill comes from an asset catalogue it inverts and the literal does not follow. Bonus: the fix is a colour that inverts with it, not a second hardcoded value.

How do you test contrast so the test stays true as the design changes?

> LISTEN FOR
>
> resolving the real catalogue colours at runtime under each appearance and contrast setting, rather than asserting values copied from a design file. Strong answers mention the drift case: the document and the assets disagreeing, with the test following the document.

Under Increase Contrast you need a card's border to become more visible. The border is white at 95% on a white card. What do you change?

> LISTEN FOR
>
> noticing that raising opacity does nothing — white on white is invisible at any alpha — so the hue has to flip to a dark line. This is the question that separates people who reason about rendered output from people who adjust numbers.

Your accessibility audit reports 42 contrast failures after a redesign, up from 4. Do you ship?

> LISTEN FOR
>
> measuring before reacting. Here the failures were system label colours that fail on any light background — 3.44:1 on pure white, before the redesign existed. A candidate who proposes a fix before establishing the baseline has told you something.

TESTING AND TOOLING

Your new test passes. How do you know it is testing anything?

> LISTEN FOR
>
> deliberately breaking the thing and watching the test fail. Ideally a story about a green run that was green because the filter matched no tests, or a fixture that made the assertion vacuous.

A performance instrument reports you are over budget, but a Release build measures slower than Debug. What do you conclude?

> LISTEN FOR
>
> suspicion of the instrument rather than the code, and a plan to measure independently — here, from kernel process start to first interactive frame, inside the app.

Which conventions in a codebase deserve to be tests rather than documentation?

> LISTEN FOR
>
> the ones that fail silently and are invisible in review: wall-clock reads, networking in an offline app, colour literals, raw animation calls. Each of those shipped a bug here before it became a test.

PRODUCT JUDGEMENT

Retention is 7% at day one, but the users who stay open the app three times a day. What do you do?

> LISTEN FOR
>
> separating "the product fails people" from "people never reach the product". The engagement figure rules out the first. Then: what sits between install and first value, and what brings someone back tomorrow.

You shipped a redesign, changed category and rewrote keywords in the same week. Conversion halved. What caused it?

> LISTEN FOR
>
> "I cannot tell", said plainly, followed by what would have made it answerable. Any confident single-cause answer is the wrong one.

Sharing progress to social media is listed as a non-goal. A user asks for it. How do you decide?

> LISTEN FOR
>
> separating the mechanism from the category: feeds, friends and leaderboards need accounts, a backend and moderation; rendering an image to the system share sheet needs none of those and does not break an offline guarantee. Also listen for the behavioural risk — an audience changes what people optimise for.

You built a decorative element, it matches the approved design, and on device it looks wrong. What now?

> LISTEN FOR
>
> willingness to remove their own work, and to restore the rule it had required an exception to. A note recording that it was tried and why it went is the difference between a decision and an accident.

SCOPE AND CONSTRAINT

This app has no accounts, no sync, no analytics SDK and no third-party dependencies. Defend that.

> LISTEN FOR
>
> the product argument rather than a purity one: it finishes, there is nothing to scroll, nothing leaves the device — and that each of those is a claim made in the App Store listing, backed by a test that fails on networking symbols.

Adding one animation library would cost about 2 MB. What else does it cost?

> LISTEN FOR

> the second-order costs: a privacy manifest to re-verify, a review-notes claim about zero dependencies that becomes false, and a motion policy that the library does not know about. The binary size is the least interesting part.

The arc runs October to December. Someone downloads it on 2 January. What is their experience?

> LISTEN FOR
>
> spotting that the product works fine — arbitrary dates are supported — while the positioning speaks entirely to people who started in October. A seasonal product has an off-season, and it is a copy problem before it is a code problem.

Winter Arc is a solo iOS project: SwiftUI, SwiftData and WidgetKit, Swift 6 strict concurrency, no third-party dependencies and no networking code. Figures in this document are measured values from the test suite and App Store Connect, not estimates.